

MyHDL: A Python Based Hardware Description Language

JAI SACHITH PAUL

Conventionally, the design of digital hardware systems is done using a specialised hardware description language (HDL), like VHDL and Verilog. This selection is based on the idea that hardware design has its own unique requirements.

MyHDL is an open source platform developed by Jan Decaluwe for using Python, a general-purpose high-level language for hardware design. A designer using this software can benefit from the power of Python language as well as the merits of a free, open source software.

The software can be installed on any platform that supports Python. The installation guide for MyHDL on a typical Linux or Unix system is given in the installation document in this month's EFY Plus DVD. Non-Linux platforms, like Windows, also have a similar installation procedure. Try out the latest release of this software, MyHDL0.8.1, in this month's DVD.

Key functionalities of the tool

The main vision behind developing this open source tool is to empower hardware engineers with the power, clarity and simplicity of Python. Let us take a look at the main functions of this software in detail.

High-level modelling. Python is a very high-level language, famous for giving elegant solutions to complex modelling problems. Apart from this, the ability of Python of rapid application development and experimentation is outstanding. All these features

Generators

We are quite familiar with Function Call in high-level programming languages. When a function is called, a private namespace is allocated to it, where local variables are created. When this called function returns a value to the main function, local variables are destroyed. If we call the same function again, it will get a fresh set of local variables. Now suppose, local variables are not destroyed on exiting the function. We could resume the function where it was left off. These kinds of functions that can be resumed are called generators.

of MyHDL make it an ideal solution for hardware modelling.

In order to model hardware concurrency, MyHDL makes use of Python generators. Generators used in MyHDL resemble the Always blocks in Verilog and processes in VHDL. The software models a hardware module as a function that returns generators. A wide range of features, such as arbitrary hierarchy, named-port association, arrays of instances and conditional instantiation, can be easily supported because of this straightforward approach. Classes in MyHDL support traditional hardware description concepts. There is a signal class to support communication between generators, a class to support bit-oriented operations and a class for enumeration types.

Converting the design to Verilog and VHDL. Although there are some limitations, designs in MyHDL can be converted into Verilog or VHDL. Of course, the convertible subset is restricted. But it is much wider than the standard synthesis subset. It includes features that can be used for high-level modelling and test benches. Also, conversion limitations are only applicable for the code inside the generators. Python's full power can be used without compromising convertibility outside the generators.



The conversion provides a path into the traditional design flow, including synthesis and implementation.

A number of complex tasks in VHDL and Verilog, such as handling of signed arithmetic issues, can be directly automated with the help of the converter in MyHDL. The converter works on a fully-elaborate, instantiated design. As a result, the original design structure can be arbitrarily complex.

Simulation and verification functions. MyHDL has a built-in simulator that runs on top of the Python interpreter, which supports waveform viewing. Tracing the changes of the signal in a VCD file enables viewing of the waveform. The software enables the Python unit test framework to be used in hardware designs.

Unit testing is a very popular verification technique in modern software engineering. But when it comes